

Day 7

Date: 24 September 2026

Topic: RAG Fundamentals

1. What I Learned

I learned the fundamentals of **Retrieval-Augmented Generation (RAG)** and how it can be used to provide relevant external information to a Large Language Model before generating an answer.

I learned the main components of a RAG pipeline:

- Document loading
- Document chunking
- Text embeddings
- Vector databases
- Similarity search
- Document retrieval
- Context construction
- LLM-based answer generation
- Grounded generation

I also learned that RAG does not depend on a specific LLM provider. In this practical task, I used **Google Gemini** for the generation stage and **ChromaDB** as the vector database.

2. What I Implemented

I implemented a basic **RAG Question Answering System** using Python.

The system contains three text documents:

data/

python.txt

machine_learning.txt

rag.txt

I implemented the following components:

Document Loading : The system reads .txt files from the data folder.

Chunking: The documents are divided into smaller text chunks so that relevant sections can be retrieved more effectively.

Embeddings: I used the Sentence Transformers model to convert text chunks into numerical vector representations.

Vector Database: I used ChromaDB to store the document chunks and their embeddings.

Retrieval: When a user asks a question, the question is converted into an embedding and ChromaDB is used to retrieve the most relevant document chunks.

Context Construction: The retrieved chunks are combined into a context that is provided to the language model.

Generation: I used the **Google Gemini API** to generate the final answer based on the retrieved context.

3. Technologies Used

Python

Sentence Transformers

ChromaDB

Google Gemini API

google-genai

python-dotenv

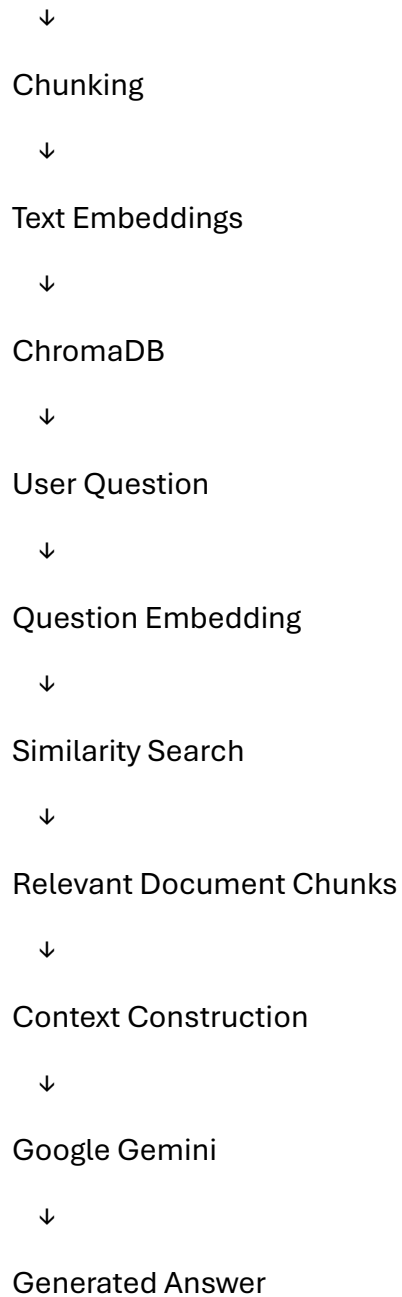
4. RAG Pipeline Implemented

The implemented pipeline is:

Documents

↓

Document Loading



5. What Worked

The document loading and chunking process worked successfully.

The text documents were converted into embeddings using Sentence Transformers.

The embeddings and document chunks were successfully stored in ChromaDB.

The vector database successfully stored:

Stored 7 chunks in ChromaDB.

The retrieval component was also implemented to search for relevant document chunks based on the user's question.

The Gemini integration was implemented to generate answers using the retrieved context.

6. What Failed

During testing, the Gemini API initially returned the following error:

503 UNAVAILABLE

The error indicated that the selected Gemini model was temporarily experiencing high demand.

I also encountered an environment-related error:

ModuleNotFoundError: No module named 'chromadb'

This occurred because ChromaDB was not installed in the Day 7 virtual environment.

7. How I Solved It

For the missing ChromaDB package, I installed the required dependency using:

```
python -m pip install chromadb
```

I also created a requirements.txt file containing the required project dependencies:

```
sentence-transformers
```

```
chromadb
```

```
python-dotenv
```

```
google-genai
```

For the Gemini API availability issue, I investigated the error and changed the Gemini model configuration to another available model rather than changing the RAG architecture.

The RAG pipeline itself was working correctly up to the generation stage because the documents were successfully embedded and stored in ChromaDB.

8. Testing Performed

I tested the system using questions related to the stored documents.

Test 1

Question:

What is RAG?

Expected result:

The system should retrieve information from rag.txt and generate an answer based on that information.

Test 2

Question:

What are the main steps of a RAG pipeline?

Expected result:

The system should retrieve the relevant RAG chunks containing information about:

- Document loading
- Chunking
- Embeddings
- Vector storage
- Retrieval
- Context construction
- Generation

Test 3

Question:

What is supervised learning?

Expected result:

The system should retrieve relevant information from machine_learning.txt.

Test 4

Question:

Why is Python used in AI?

Expected result:

The system should retrieve relevant information from python.txt.

```
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\Week-02\Day-07\practic
al-task> python vector_store.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
Loading weights: 100%|██████████████████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 1720.82it/s]
Stored 7 chunks in ChromaDB.
```

```
al-task> & "e:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-01\Day-02\.venv\Scripts\Activate.ps1"
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-02\Day-07\practic
al-task> python documents.py

Document: machin_learning.txt
Number of chunks: 2

Chunk 1:
Machine learning is a branch of artificial intelligence that allows computers to learn patterns from data.

A machine learning model is trained using data and can make predictions or decisions based on patterns learned during t
raining.

Common types of machine learning include supervised learning, u

Chunk 2:
unsupervised learning, and reinforcement learning.

Supervised learning uses labelled data to train models for tasks such as classification and regression.

Unsupervised learning works with data that does not have predefined labels. It can be used to discover patterns or grou
ps in data.

Document: python.txt
Number of chunks: 2

Chunk 1:
python is a high-level programming language known for its simple and readable syntax
```

```
al-task> python vector_store.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
Loading weights: 100%|██████████████████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 742.40it/s]
Stored 7 chunks in ChromaDB.
```

```
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-02\Day-07\practic
al-task> python retriever.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
Loading weights: 100%|██████████████████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 1711.00it/s]
Enter your question:
```



```
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-02\Day-07\practical-task> python app.py
```

Retrieval-Augmented Generation, commonly called RAG, is a technique that combines information retrieval with text generation.

Question: Question: What are the main steps of a RAG pipeline?

Retrieved Context:

Source: rag.txt

del uses the retrieved context to generate an answer to the user's question.

A typical RAG pipeline contains document loading, chunking, embedding, vector storage, retrieval, context construction, and generation.

RAG can help applications answer questions using information from specific documents

Source: rag.txt

rather than relying only on the language model's internal knowledge.

The main components of a RAG system are document ingestion, chunking, embeddings, a vector database, similarity search, context construction, and language model generation.

Source: rag.txt

Retrieval-Augmented Generation, commonly called RAG, is a technique that combines information retrieval with text generation.

A RAG system first retrieves relevant information from a collection of documents. The retrieved information is then provided to a language model as context.

The language mo

```
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-02\Day-07\practical-task> python app.py
```

Type 'exit' to stop.

Question: What is RAG?

Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.

Retrieved Context:

Source: rag.txt

rather than relying only on the language model's internal knowledge.

The main components of a RAG system are document ingestion, chunking, embeddings, a vector database, similarity search, context construction, and language model generation.

Source: rag.txt

Retrieval-Augmented Generation, commonly called RAG, is a technique that combines information retrieval with text generation.

A RAG system first retrieves relevant information from a collection of documents. The retrieved information is then provided to a language model as context.

The language mo

The language mo

Source: rag.txt

del uses the retrieved context to generate an answer to the user's question.

A typical RAG pipeline contains document loading, chunking, embedding, vector storage, retrieval, context construction, and generation.

RAG can help applications answer questions using information from specific documents

Generated Answer:

Based on the provided context, RAG stands for Retrieval-Augmented Generation. It is a technique that combines information retrieval with text generation. In a RAG system, relevant information is first retrieved from a collection of documents and then provided to a language model as context to generate an answer to the user's question.

Question: []

9. Problems Faced

Gemini API 503 Error

The Gemini API returned:

503 UNAVAILABLE

The message indicated that the selected model was experiencing high demand.

```
(.venv) PS E:\VisionRoot-AI engineer Internship\Github\AI-Engineer-Training\AI-Engineer-Training\Week-02\Day-07\practical-task> python app.py

=====
GEMINI RAG QUESTION ANSWERING SYSTEM
=====

GEMINI RAG QUESTION ANSWERING SYSTEM
=====

Ask a question about the documents.
Type 'exit' to stop.

Ask a question about the documents.
Type 'exit' to stop.

Type 'exit' to stop.

Question: What is RAG?
Question: What is RAG?
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.

An error occurred:
503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.', 'status': 'UNAVAILABLE'}}
Question:
Traceback (most recent call last):
```

ChromaDB Missing

The following error occurred:

ModuleNotFoundError: No module named 'chromadb'

10. What I Would Improve

If I continue improving this project, I would:

- Implement better chunking with chunk overlap.
- Experiment with different chunk sizes.

- Add more documents to the knowledge base.
- Improve retrieval accuracy.
- Add similarity scores to retrieved results.
- Add source citations to generated answers.
- Add conversation history.
- Add a user interface.
- Add error handling for Gemini API failures.
- Implement retry logic for temporary API errors.
- Evaluate retrieval quality using different questions.
- Experiment with different embedding models.
- Add metadata filtering to ChromaDB.